Reporte

| | |
|---|---|
| **Nombre:**<br>1. Roberto Santana Trujillo<br>2. Isaías Leonardo Salazar Torres | **Matrícula:**<br>2227084<br>2227016 |
| **Nombre del curso:** Componentes de lógica de negocio | **Nombre del profesor:** Miguel Ángel Gómez Marroquín |
| **Módulo Netacad:** 4 | **Actividad:** AF4 Desarrollo del núcleo de negocio TechStore |
| **Fecha de entrega:** 28/10/2026 | |
| **Bibliografía (Formato APA)**<br>Material U4 proporcionado por el profesor y documentación técnica. Referencias completas al final del informe. | |

## Objetivo

Implementar el núcleo de negocio de TechStore a partir de los CSV y las reglas de la Unidad 4. Se desarrollaron entidades y Value Objects con invariantes, Specifications combinables, una máquina de estados con Domain Events y una API REST. La comprobación obtuvo 15 de 15 escenarios verificados y 90 pruebas aprobadas.

Este informe recopila los ocho entregables de la práctica, las cinco preguntas del reporte y la reflexión. El código ejecutable, los archivos originales y las evidencias completas se conservan en el repositorio.

**Link del repositorio en GitHub**
https://github.com/Santana-R/techstore-u4

## Procedimiento y recopilación de entregables

Se utilizó el caso común TechStore de la práctica guiada. El informe U3 se tomó como referencia de formato. La implementación conserva los cinco CSV y cargar_csv.py del paquete del profesor; no incorpora base de datos ni control de inventario.

| Entregable solicitado | Ubicación en este informe |
|---|---|
| 1. Repositorio GitHub | Sección 1 y enlace de la primera página |
| 2. README.md | Sección 2 con su contenido completo |
| 3. Captura de los 16 pedidos | Sección 3 y salida reproducible |
| 4. Bitácora E01 a E15 | Sección 4 con precondiciones, operación, resultado, evento y prueba |
| 5. pytest y resultados | Sección 5 con captura y cobertura |
| 6. Capturas de Swagger | Sección 6 con 200, 409, 422 y 404 |
| 7. Estados y Domain Events | Sección 7 con diagrama exportado desde draw.io y explicación |
| 8. Reflexión de tres errores | Sección 8 con solución y aprendizaje |

Las capturas de carga y pytest muestran visores locales de la salida real guardada en archivos de texto. Las capturas de Swagger provienen de peticiones ejecutadas en /docs. Los recortes solo encuadran las evidencias; los originales completos permanecen en evidencias/.

## 1 Repositorio y organización del dominio

El repositorio contiene app, tests, datos, diagramas, evidencias y reporte. .gitignore excluye el entorno virtual y los archivos de caché. El flujo automatizado de GitHub ejecuta carga, escenarios y pytest con Python 3.12.

Order protege el agregado mediante métodos: no expone setters de estado ni listas mutables de artículos o eventos. Money y OrderItem son inmutables; las Specifications evalúan elegibilidad sin efectos secundarios. FastAPI traduce los resultados y las excepciones a HTTP, mientras el dominio permanece independiente de la interfaz.

# 2 README completo del repositorio

Se transcribe el README.md que acompaña al código. Sus comandos permiten reproducir la actividad desde un equipo nuevo.

## TechStore U4

Laboratorio de Componentes de Lógica de Negocio de FIME UANL. Implementa pedidos, reglas de descuento, una máquina de estados, eventos de dominio y una API REST sobre los CSV originales del paquete del alumno. La validación produjo 90 pruebas aprobadas y 15 de 15 escenarios verificados.

Integrantes: Roberto Santana Trujillo (2227084) e Isaías Leonardo Salazar Torres (2227016). Profesor: Miguel Ángel Gómez Marroquín.

Repositorio: https://github.com/Santana-R/techstore-u4

## Instalación

Requiere Python 3.11 o superior; se verificó con Python 3.12.14. Clona el repositorio y entra a su carpeta.

```
git clone https://github.com/Santana-R/techstore-u4.git
cd techstore-u4
python -m venv .venv
```

Activa el entorno según tu sistema. En macOS o Linux:

```
source .venv/bin/activate
```

En Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

En Windows CMD:

```
.venv\Scripts\activate.bat
```

Instala las versiones verificadas:

```
python -m pip install -r requirements.txt
```

Si PowerShell bloquea la activación, usa directamente `.venv\Scripts\python.exe` en lugar de `python`. En macOS o Linux, usa `.venv/bin/python`. La instalación ocurre dentro del entorno virtual.

## Ejecución y comprobación

Ejecuta desde la raíz del proyecto:

```
python cargar_csv.py
python ejecutar_escenarios.py
python -m pytest -q
python -m uvicorn app.api:app --reload
```

El cargador muestra los 16 pedidos en DRAFT. O001 debe tener 4 unidades y 2100.00 MXN. El ejecutor verifica E01 a E15 con datos nuevos por caso y actualiza `evidencias/escenarios.json` y `evidencias/bitacora.csv`. pytest debe informar 90 pruebas aprobadas.

Swagger se abre en http://127.0.0.1:8000/docs y el contrato OpenAPI está en http://127.0.0.1:8000/openapi.json. Las capturas del reporte usaron el puerto 8764; es la misma aplicación. Detén el servidor con Ctrl+C. Al reiniciarlo, todos los pedidos vuelven a DRAFT.

## Estructura

```
app/domain.py          Entidades, VO, estados y eventos
app/specifications.py  Reglas AND, OR y NOT; descuentos
app/api.py             Adaptador HTTP y aplicación FastAPI
datos/                 Los cinco CSV originales del inge
tests/                 Dominio, reglas, API y escenarios
diagramas/             Draw.io editable y su PNG exportado
evidencias/            Bitácora, logs, OpenAPI y capturas
cargar_csv.py          Cargador original del paquete
ejecutar_escenarios.py  Ejecución reproducible de E01 a E15
requirements.txt       Versiones verificadas del entorno
```

`.venv`, `__pycache__` y `.pytest_cache` se excluyen del repositorio. Los CSV y el cargador se conservaron sin cambios.

## Reglas de negocio

• Money es inmutable, utiliza Decimal y rechaza importes negativos, no finitos y valores float. add exige la misma moneda. Los pedidos usan MXN.

• OrderItem es inmutable y exige SKU y cantidad entera positiva. El máximo es 20 unidades sumadas, aunque estén en uno o varios renglones.

• Descuento del 10 por ciento: cliente VIP, total bruto estrictamente mayor que 500 MXN, al menos 4 unidades y sin promoción.

• Descuento del 5 por ciento: total bruto estrictamente mayor que 5000 MXN y sin promoción. Si ambas reglas se cumplen, se aplica solo 10 por ciento.

• El descuento se calcula sobre el total bruto. El importe descontado se redondea a centavos con ROUND_HALF_UP; consultar el descuento no modifica el pedido ni genera eventos.

• Confirmar exige DRAFT, artículos y cliente activo. Enviar exige CONFIRMED y guía. Cancelar exige CONFIRMED y motivo. Entregar y devolver exigen SHIPPED.

• Toda transición válida registra un evento con pedido, cliente, total y fecha UTC. Un rechazo conserva el estado, los artículos y los eventos existentes.

• La existencia de productos es contexto del catálogo. No hay control de inventario, base de datos ni envío real de mensajes. Los objetos se guardan en memoria y los eventos se consultan en la respuesta de cada pedido.

## Endpoints

Todos los métodos devuelven 200 cuando la operación es válida. Los importes se serializan como cadenas decimales para conservar su representación.

| Método | Ruta | Entrada o resultado |
|---|---|---|
| GET | /orders/{order_id} | Consulta el pedido y sus eventos |
| POST | /orders/{order_id}/confirm | Confirma sin cuerpo JSON |
| POST | /orders/{order_id}/ship | tracking_number como cadena |
| POST | /orders/{order_id}/cancel | reason como cadena |
| POST | /orders/{order_id}/deliver | Entrega desde SHIPPED |
| POST | /orders/{order_id}/return | Devuelve desde SHIPPED |
| POST | /orders/{order_id}/items | sku y quantity; precio del CSV |

404 identifica pedido o SKU inexistente; 409 indica una transición prohibida; 422 indica datos o una regla de negocio inválidos. Los cuerpos rechazan campos adicionales y cantidades que no sean enteros positivos.

## Ejemplo en Swagger

1. En POST /orders/{order_id}/confirm, pulsa Try it out y escribe O013. Execute devuelve 200, CONFIRMED y OrderConfirmed.

2. Ejecuta otra vez el mismo pedido: devuelve 409 y conserva un solo evento.

3. Usa O007 para comprobar 422 por pedido vacío y NOEXISTE para comprobar 404.

4. Reinicia el servidor para repetir los casos desde DRAFT. Las pruebas automáticas crean una aplicación independiente por caso.

## Evidencias y diagrama

`evidencias/carga.txt`, `pytest.txt` y `escenarios.txt` conservan la salida real de ejecución. Sus capturas se muestran en visores locales; los HTML identifican expresamente el comando y el archivo de origen. Las imágenes swagger_200, swagger_404, swagger_409 y swagger_422 son capturas reales de /docs.

El archivo `diagramas/techstore_estados.drawio` es editable en https://app.diagrams.net. Su PNG se exportó desde draw.io e incluye una copia editable del diagrama. Los recortes del reporte solo encuadran los resultados; las capturas completas se conservan en el repositorio.

El reporte único se encuentra en `reporte/2227084_2227016_AF4_CLNEG_TechStore.pdf`. Incluye los ocho entregables, la bitácora completa, las cinco preguntas resueltas y la reflexión.

## Referencias

• Material del curso: práctica guiada TechStore, manual ilustrado U4 y paquete de CSV y cargador proporcionados por el profesor.

• FastAPI: https://fastapi.tiangolo.com/tutorial/testing/ y https://fastapi.tiangolo.com/tutorial/handling-errors/

• pytest: https://docs.pytest.org/en/stable/how-to/parametrize.html

• Python Decimal: https://docs.python.org/3/library/decimal.html

## 3 Carga exitosa de los 16 pedidos

Se cargaron los 16 pedidos desde los CSV originales. Todos comienzan DRAFT; O007 se conserva vacío para comprobar su rechazo al confirmar.

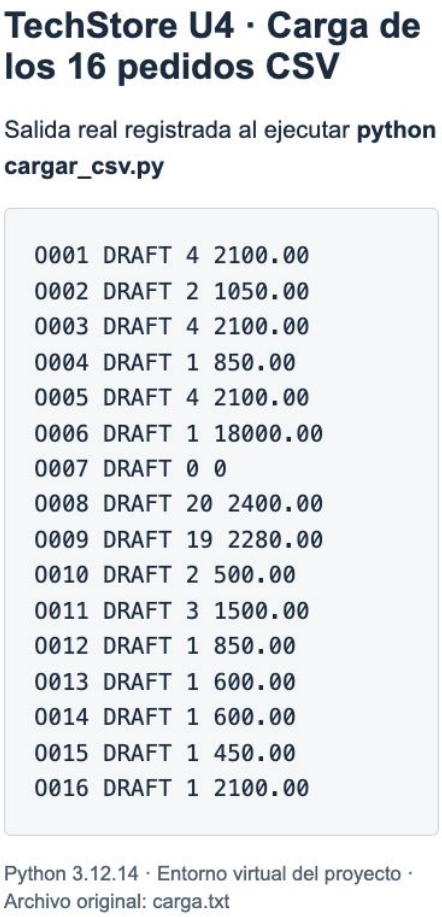



Figura 1. Captura del visor local de la salida real de cargar_csv.py. El archivo de origen es evidencias/carga.txt.

| Pedido | Cliente | Unidades | Cálculo manual en MXN |
|---|---|---|---|
| O001 | C001 | 4 | 2 × 600 + 2 × 450 = 2100.00 |
| O006 | C006 | 1 | 1 × 18000 = 18000.00 |
| O010 | C010 | 2 | 1 × 120 + 1 × 380 = 500.00 |

# 4 Bitácora de escenarios E01 a E15

Cada escenario se ejecutó con load_fixture() nuevo. La prueba parametrizada tests/test_scenarios.py verifica resultados concretos y comprueba que una operación rechazada conserve estado, artículos y eventos.

## E01 Pedido O001

**Precondiciones:** DRAFT; cliente C001, VIP, activo; 4 unidades; total 2100.00 MXN; sin promoción.

**Operación:** evaluar_descuento.

**Resultado real:** DRAFT. Cumple VIP, total >500, al menos 4 unidades y ausencia de promoción. El descuento es 210.00 y el neto 1890.00 MXN.

**Evento:** Ninguno.

**Prueba:** `test_escenario[E01]` aprobada.

## E02 Pedido O002

**Precondiciones:** DRAFT; cliente C002, no VIP, activo; 2 unidades; total 1050.00 MXN; sin promoción.

**Operación:** evaluar_descuento.

**Resultado real:** DRAFT. El cliente no es VIP y 1050.00 no supera 5000. No se concede descuento.

**Evento:** Ninguno.

**Prueba:** `test_escenario[E02]` aprobada.

## E03 Pedido O003

**Precondiciones:** DRAFT; cliente C003, VIP, inactivo; 4 unidades; total 2100.00 MXN; sin promoción.

**Operación:** confirmar.

**Resultado real:** InvalidOrderData; DRAFT. El cliente C003 está inactivo. Se rechaza confirmar y el pedido sigue en DRAFT.

**Evento:** Ninguno. El paso rechazado no añadió evento.

**Prueba:** `test_escenario[E03]` aprobada.

# 4 Bitácora de escenarios continuación

## E04 Pedido O005

**Precondiciones:** DRAFT; cliente C005, VIP, activo; 4 unidades; total 2100.00 MXN; con promoción.

**Operación:** evaluar_descuento.

**Resultado real:** DRAFT. La promoción activa bloquea ambos descuentos, aunque el cliente sea VIP y reúna monto y unidades.

**Evento:** Ninguno.

**Prueba:** `test_escenario[E04]` aprobada.

## E05 Pedido O006

**Precondiciones:** DRAFT; cliente C006, no VIP, activo; 1 unidades; total 18000.00 MXN; sin promoción.

**Operación:** evaluar_descuento.

**Resultado real:** DRAFT. 18000.00 supera 5000 y no hay promoción. Descuento de 900.00 y neto de 17100.00 MXN.

**Evento:** Ninguno.

**Prueba:** `test_escenario[E05]` aprobada.

## E06 Pedido O007

**Precondiciones:** DRAFT; cliente C007, VIP, activo; 0 unidades; total 0.00 MXN; sin promoción.

**Operación:** confirmar.

**Resultado real:** InvalidOrderData; DRAFT. No tiene artículos. La confirmación se rechaza sin generar evento.

**Evento:** Ninguno. El paso rechazado no añadió evento.

**Prueba:** `test_escenario[E06]` aprobada.

# 4 Bitácora de escenarios continuación

## E07 Pedido O008

**Precondiciones:** DRAFT; cliente C008, no VIP, activo; 20 unidades; total 2400.00 MXN; sin promoción.

**Operación:** confirmar.

**Resultado real:** CONFIRMED. Las 20 unidades exactas están permitidas. Confirmar cambia el estado a CONFIRMED.

**Evento:** OrderConfirmed

**Prueba:** `test_escenario[E07]` aprobada.

## E08 Pedido O009

**Precondiciones:** DRAFT; cliente C009, VIP, activo; 19 unidades; total 2280.00 MXN; sin promoción.

**Operación:** agregar SKU003 x2.

**Resultado real:** OrderItemLimitExceeded; DRAFT. 19 + 2 serían 21 unidades. Se rechaza antes de añadir: se conservan 19 unidades y 2280.00 MXN.

**Evento:** Ninguno. El paso rechazado no añadió evento.

**Prueba:** `test_escenario[E08]` aprobada.

## E09 Pedido O010

**Precondiciones:** DRAFT; cliente C010, no VIP, activo; 2 unidades; total 500.00 MXN; sin promoción.

**Operación:** evaluar_descuento.

**Resultado real:** DRAFT. El total es exactamente 500.00; no cumple el umbral estricto >500 ni la alternativa >5000.

**Evento:** Ninguno.

**Prueba:** `test_escenario[E09]` aprobada.

# 4 Bitácora de escenarios continuación

## E10 Pedido O011

**Precondiciones:** DRAFT; cliente C011, VIP, activo; 3 unidades; total 1500.00 MXN; sin promoción.

**Operación:** evaluar_descuento.

**Resultado real:** DRAFT. Es VIP y supera 500, pero tiene 3 unidades. No cumple el mínimo de 4 y tampoco supera 5000.

**Evento:** Ninguno.

**Prueba:** `test_escenario[E10]` aprobada.

## E11 Pedido O012

**Precondiciones:** DRAFT; cliente C012, no VIP, activo; 1 unidades; total 850.00 MXN; sin promoción.

**Operación:** confirmar y enviar sin guia.

**Resultado real:** InvalidOrderData; CONFIRMED. Confirmar es válido y emite OrderConfirmed. Enviar sin guía se rechaza: queda CONFIRMED y se conserva únicamente ese evento.

**Evento:** OrderConfirmed El paso rechazado no añadió evento.

**Prueba:** `test_escenario[E11]` aprobada.

## E12 Pedido O013

**Precondiciones:** DRAFT; cliente C001, VIP, activo; 1 unidades; total 600.00 MXN; sin promoción.

**Operación:** confirmar dos veces.

**Resultado real:** InvalidOrderTransition; CONFIRMED. La primera confirmación es válida. La segunda se rechaza desde CONFIRMED y no duplica OrderConfirmed.

**Evento:** OrderConfirmed El paso rechazado no añadió evento.

**Prueba:** `test_escenario[E12]` aprobada.

# 4 Bitácora de escenarios continuación

## E13 Pedido O014

**Precondiciones:** DRAFT; cliente C002, no VIP, activo; 1 unidades; total 600.00 MXN; sin promoción.

**Operación:** confirmar, enviar y cancelar.

**Resultado real:** InvalidOrderTransition; SHIPPED. Confirmar y enviar con guía son válidos. Cancelar desde SHIPPED se rechaza; conserva los dos eventos anteriores.

**Evento:** OrderConfirmed → OrderShipped El paso rechazado no añadió evento.

**Prueba:** `test_escenario[E13]` aprobada.

## E14 Pedido O015

**Precondiciones:** DRAFT; cliente C005, VIP, activo; 1 unidades; total 450.00 MXN; sin promoción.

**Operación:** confirmar, enviar y devolver.

**Resultado real:** RETURNED. Confirmar, enviar y devolver son válidos. La regla didáctica permite devolver desde SHIPPED y termina en RETURNED.

**Evento:** OrderConfirmed → OrderShipped → OrderReturned

**Prueba:** `test_escenario[E14]` aprobada.

## E15 Pedido O016

**Precondiciones:** DRAFT; cliente C007, VIP, activo; 1 unidades; total 2100.00 MXN; sin promoción.

**Operación:** confirmar, enviar y entregar.

**Resultado real:** DELIVERED. Confirmar, enviar y entregar son válidos. Termina en DELIVERED con tres eventos de la secuencia.

**Evento:** OrderConfirmed → OrderShipped → OrderDelivered

**Prueba:** `test_escenario[E15]` aprobada.

## 5 Pruebas automáticas y resultados

El resultado registrado fue 90 passed in 0.54s. Se verifican tanto las operaciones válidas como la ausencia de cambios parciales cuando una regla o transición falla. No se modificaron los CSV ni los resultados esperados para hacer pasar los casos.

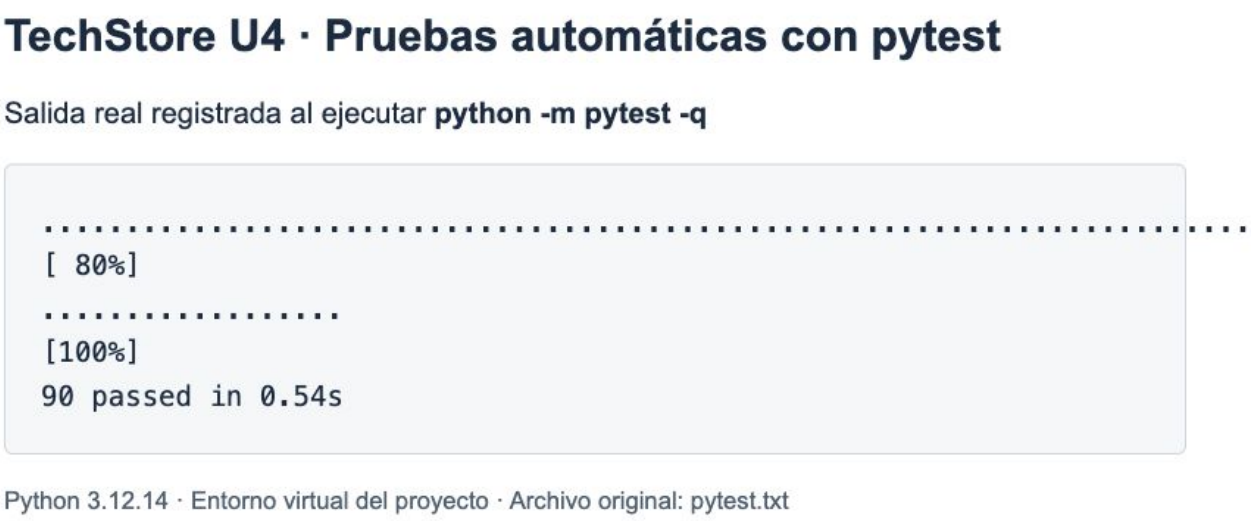



Figura 2. Captura del visor de salida real de python -m pytest -q. El registro completo se conserva en evidencias/pytest.txt.

| Archivo | Pruebas | Comportamientos comprobados |
|---|---|---|
| test_domain.py | 52 | Money, cantidad, carga, máximo de unidades, inmutabilidad, guía y motivo; matriz de 6 estados × 5 operaciones. |
| test_specifications.py | 11 | Reglas independientes, AND/OR/NOT, 500 y 5000 exactos, promoción, prioridad del 10 por ciento y redondeo. |
| test_scenarios.py | 15 | E01 a E15 con resultado, estado y secuencia de eventos esperados. |
| test_api.py | 12 | HTTP 200/404/409/422, contrato OpenAPI, JSON estricto, estados y aplicaciones independientes. |
| Total | 90 | Todas aprobadas en la ejecución adjunta. |

La prueba de inventario confirma que la existencia del catálogo no limita el pedido. La frontera de 20 unidades se conserva. También se verifica que un pedido con 19 unidades no reciba dos adicionales y que las listas de artículos y eventos no puedan modificarse desde fuera del agregado.

Los descuentos se prueban sobre el importe bruto. Cuando VIP y monto >5000 cumplen ambas reglas, el resultado es solo 10 por ciento. En presencia de promoción, ambas quedan excluidas.

# 6 Evidencias de Swagger operación exitosa

En /docs se ejecutó POST /orders/O012/confirm sobre un pedido DRAFT con un artículo y cliente activo. La respuesta fue 200: status CONFIRMED y un evento OrderConfirmed. El total permanece en 850.00 MXN.

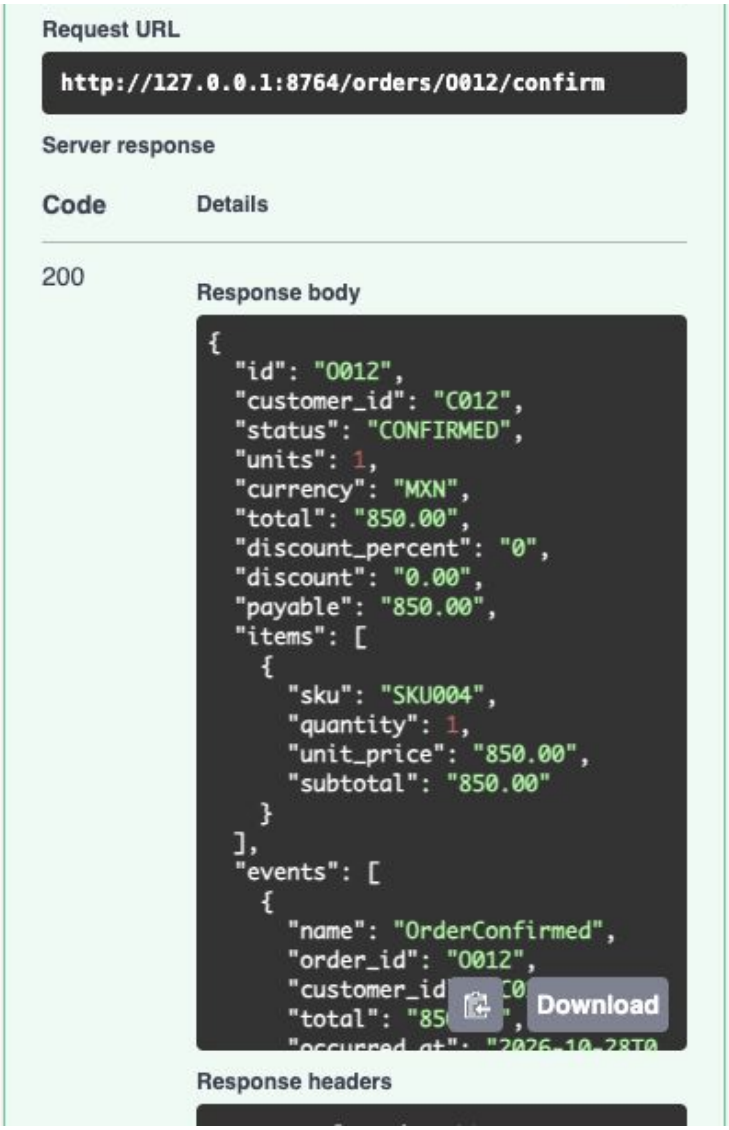



Figura 3. Recorte de la captura real de Swagger. Se observan URL, código HTTP, estado y evento de confirmación. Captura completa: evidencias/swagger_200.png.

El objeto de respuesta incluye unidades, moneda, total bruto, porcentaje, descuento, neto, artículos y eventos. Los importes viajan como cadenas decimales. La consulta GET /orders/O012 permite recuperar de nuevo el estado resultante.

# 6 Evidencias de Swagger operaciones rechazadas

La misma confirmación sobre O012 se ejecutó por segunda vez. La respuesta 409 expresa una transición inválida. La prueba de API confirma que el pedido y sus eventos permanecen iguales después del rechazo.




Figura 4. HTTP 409 al confirmar de nuevo O012. Requiere DRAFT y el pedido ya está CONFIRMED.

Se intentó confirmar O007, que no tiene artículos. La respuesta 422 señala que los datos del pedido no permiten la operación; se conserva DRAFT sin eventos.




Figura 5. HTTP 422 por pedido vacío. La misma política impide confirmar al cliente inactivo O003.

# 6 Evidencias de Swagger identificador inexistente

POST /orders/NOEXISTE/confirm devolvió 404 con el mensaje Pedido inexistente. La API responde con un error comprensible en lugar de crear un pedido o convertirlo en un error 500.




Figura 6. HTTP 404 observado en Swagger para un identificador que no existe en los CSV.

| Situación | HTTP | Efecto en el dominio |
|---|---|---|
| Confirmación válida | 200 | Cambia estado y añade OrderConfirmed. |
| Pedido inexistente | 404 | No se invoca una transición de un pedido. |
| Confirmación repetida | 409 | Conserva estado, artículos y eventos. |
| Pedido vacío o cliente inactivo | 422 | Permanece DRAFT y no registra evento. |

La API delega las reglas al dominio. InvalidOrderTransition se traduce a 409; los demás rechazos de negocio a 422. Los datos de cuerpo inválidos también se comunican como 422. El contrato completo se entrega en evidencias/openapi.json.

# 7 Diagrama de estados y Domain Events

El diagrama se abrió y exportó en draw.io. El archivo fuente techstore_estados.drawio permanece editable en diagramas/. Las flechas muestran operaciones permitidas, condiciones y eventos. Los estados finales no tienen salidas en este laboratorio.

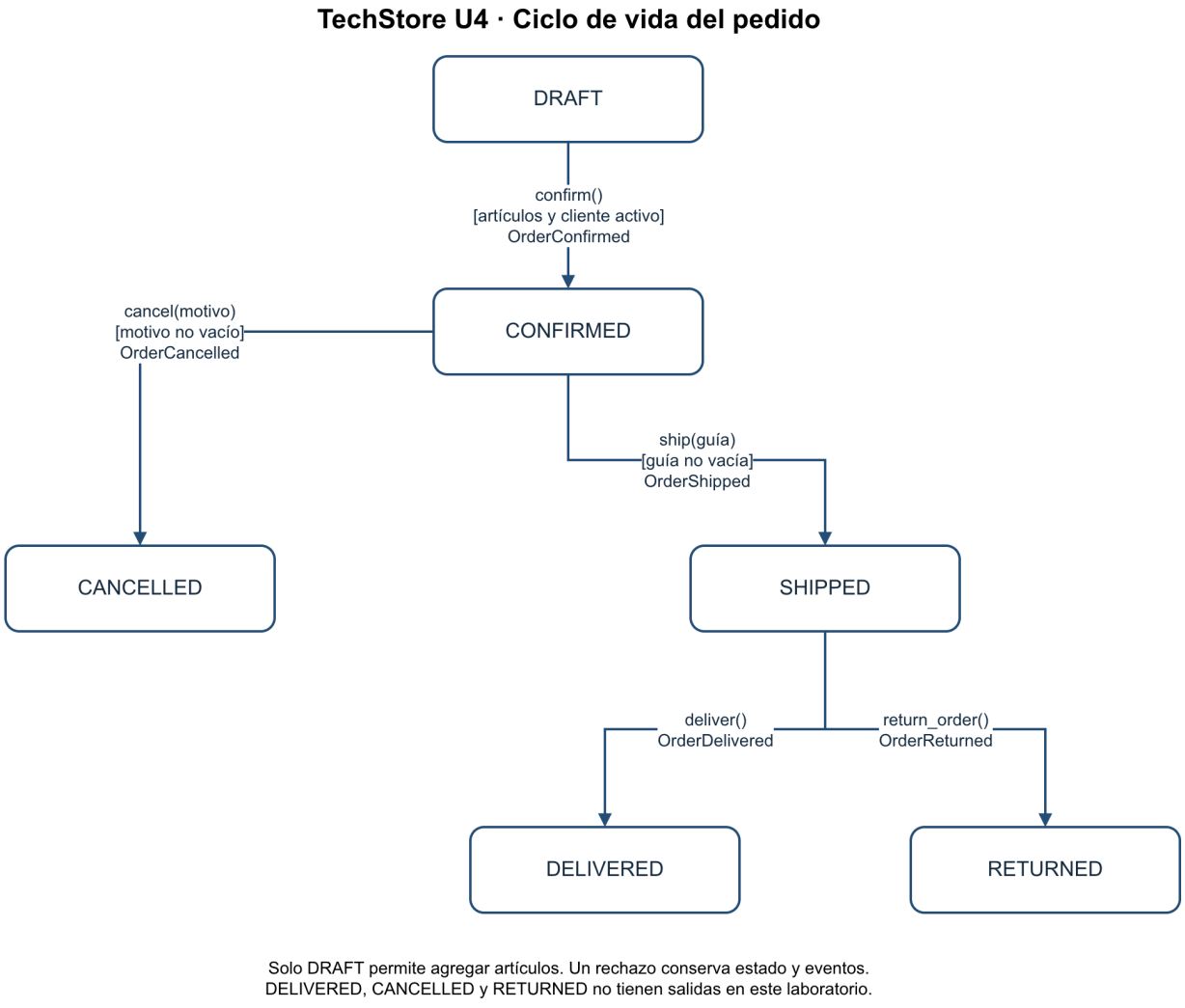



Figura 7. Máquina de estados de Order exportada desde draw.io. La devolución se admite desde SHIPPED, conforme al material del inge.

El agregado valida primero. Después actualiza el estado y añade el evento inmutable a su historial. Un error de validación no produce un hecho nuevo. Los eventos incluyen order_id, customer_id, total y occurred_at en UTC; los de envío y cancelación agregan guía o motivo.

Los eventos se almacenan dentro del pedido en memoria. Esta actividad no publica mensajes externos ni envía correo. En una ampliación, un adaptador podría consumirlos sin trasladar las reglas a FastAPI.

# 7 Transiciones y significado de los eventos

| Operación | Origen y destino | Condición y evento |
|---|---|---|
| confirm() | DRAFT → CONFIRMED | Artículos y cliente activo. Registra OrderConfirmed. |
| ship(guía) | CONFIRMED → SHIPPED | Guía no vacía. Registra OrderShipped. |
| cancel(motivo) | CONFIRMED → CANCELLED | Motivo no vacío. Registra OrderCancelled. |
| deliver() | SHIPPED → DELIVERED | Pedido enviado. Registra OrderDelivered. |
| return_order() | SHIPPED → RETURNED | Pedido enviado. Registra OrderReturned. |

OrderConfirmed comunica que la confirmación ya ocurrió. OrderShipped registra el envío y su guía. OrderCancelled deja constancia del motivo. OrderDelivered representa la entrega y OrderReturned la devolución autorizada por el flujo del laboratorio. Son hechos en pasado, no órdenes para ejecutar una acción.

E11 conserva OrderConfirmed cuando falla el envío sin guía. E12 no duplica ese evento al confirmar otra vez. E13 conserva OrderConfirmed y OrderShipped, pero no añade OrderCancelled. E14 y E15 terminan con tres eventos, uno por cada transición válida.

La matriz de pruebas cubre 30 combinaciones de estado y operación. Solo se aceptan las cinco transiciones de la tabla. Para cada rechazo se compara el estado completo antes y después; para cada éxito se comprueban el destino y el evento recién añadido.

El agregado también protege la composición: agregar artículos solo se permite en DRAFT, los precios deben estar en MXN y la suma de cantidades no puede superar 20. El descuento se evalúa mediante Specifications y no cambia el estado.

# 8 Reflexión sobre errores y aprendizajes

Durante la validación se observaron rechazos de negocio y un problema del entorno de ejecución. Los rechazos esperados forman parte de la evidencia: resolverlos significa controlar y comunicar la operación inválida, conservando la regla del laboratorio.

## Primer error exceder el límite de unidades

En E08, O009 tenía 19 unidades y se intentaron agregar dos. La operación lanzó OrderItemLimitExceeded. La solución consiste en validar la suma antes de modificar la lista y solicitar una cantidad que no exceda el máximo. La prueba confirmó que se conservaron 19 unidades y el importe previo. El aprendizaje es distinguir la cantidad de renglones de la suma de unidades.

## Segundo error repetir una confirmación

En E12 y en Swagger se llamó confirm() sobre un pedido ya CONFIRMED. Se obtuvo InvalidOrderTransition y HTTP 409. La solución fue proteger el origen permitido y no emitir otro evento. Para repetir una prueba desde el inicio se usa un fixture nuevo o se reinicia el servidor. El aprendizaje es que ejecutar un comando dos veces requiere considerar el ciclo de vida del pedido.

## Tercer error ejecutar pytest con otro intérprete

Al capturar resultados se invocó el intérprete base en lugar del ejecutable del entorno virtual y apareció No module named pytest. Se corrigió usando directamente .venv/bin/python y las dependencias de requirements.txt. La ejecución final produjo 90 pruebas aprobadas. El aprendizaje es comprobar qué intérprete ejecuta el comando, además de verificar que el paquete esté instalado.

La actividad permitió comprobar que los métodos del dominio son el punto común entre scripts, pruebas y API. La asistencia de IA se utilizó para implementar y documentar; los resultados de funcionamiento se contrastaron con ejecuciones y pruebas reproducibles.

# 9 Preguntas del reporte

**¿Por qué se usa Decimal en lugar de float para precios?**

Decimal representa los importes decimales sin el error de aproximación binaria típico de float. Se construye desde cadenas, como Decimal("600.00"). Esto permite comparar los umbrales y calcular subtotales de forma consistente. El descuento se redondea explícitamente a centavos con ROUND_HALF_UP.

**¿Cuál es la diferencia entre contar renglones y sumar unidades?**

Un renglón identifica un artículo con una cantidad. Dos renglones con diez unidades cada uno contienen veinte unidades. El máximo del laboratorio se verifica con sum(item.quantity), no con len(items). E08 muestra que añadir dos a un pedido de diecinueve debe rechazarse aunque haya pocos renglones.

**¿Qué ventaja tiene encapsular una regla en Specification?**

Permite nombrar, probar y reutilizar condiciones sin modificar el pedido. MinimumAmountSpec, VIPCustomerSpec, MinimumUnitsSpec y ActivePromotionSpec se componen con AND, OR y NOT. Así se separa la elegibilidad de la operación que calcula el importe del descuento, y la misma política puede usarse desde distintas interfaces.

**¿Por qué los eventos se registran solo después de una transición válida?**

Un evento anuncia un hecho que ya ocurrió. Registrarlo antes de validar podría comunicar un envío sin guía o una confirmación rechazada. La operación valida, cambia el estado y añade el evento. Si falla, conserva el historial anterior. E11, E12 y E13 comprueban expresamente esa ausencia de eventos nuevos.

**¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?**

Se reutilizarían app/domain.py y app/specifications.py, junto con sus pruebas, porque no dependen de HTTP. Otra interfaz llamaría a confirm(), ship() o quote_discount() con los mismos objetos. Se sustituirían los modelos de entrada y salida y la traducción de excepciones que actualmente pertenecen a app/api.py.

## Conclusiones

TechStore convierte los datos de los CSV en objetos que protegen su validez y sus operaciones. Money y OrderItem evitan valores inválidos desde su construcción; Order conserva las reglas de unidades, el estado y los eventos. Las Specifications expresan la política de descuentos sin alterar el pedido.

Los escenarios E01 a E15 comprobaron tanto los éxitos como los rechazos previstos. Las 90 pruebas verificaron las fronteras de monto y cantidad, las transiciones, los eventos y las respuestas de la API. La evidencia incluye la carga de los 16 pedidos, salidas registradas, capturas de Swagger y el diagrama editable.

La API REST ofrece una entrada al mismo dominio que utilizan el cargador, los escenarios y pytest. La persistencia en memoria es suficiente para el objetivo del laboratorio: las reglas y su comprobación son reproducibles, y todos los pedidos vuelven a DRAFT al iniciar una ejecución nueva.

### Bibliografía en formato APA

Gómez Marroquín, M. Á. (s. f.). Práctica guiada Actividad 4 Implementación del núcleo de negocio TechStore [Material de clase]. FIME, UANL.

Gómez Marroquín, M. Á. (s. f.). Manual de lectura ilustrado Unidad 4 Desarrollo de Componentes de Negocio [Material de clase]. FIME, UANL.

FIME, UANL. (s. f.). CLN U4 Paquete del alumno [Archivos CSV y cargador Python]. Material proporcionado en el curso.

FastAPI. (s. f.). Testing. https://fastapi.tiangolo.com/tutorial/testing/

FastAPI. (s. f.). Handling Errors. https://fastapi.tiangolo.com/tutorial/handling-errors/

pytest. (s. f.). How to parametrize fixtures and test functions. https://docs.pytest.org/en/stable/how-to/parametrize.html

Python Software Foundation. (s. f.). decimal Decimal fixed point and floating point arithmetic. https://docs.python.org/3/library/decimal.html